CUDA C++ By Example

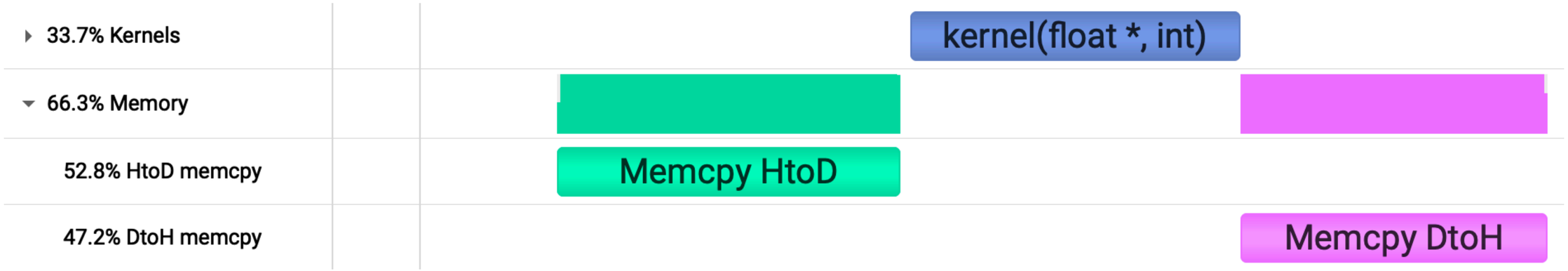
More CUDA Streams

```
while (true) {
    read_sensor_data(h_sensor_data);
    cudaMemcpy(d_sensor_data, h_sensor_data, ... );
    kernel<<< grid, block >>>(d_control_data, d_sensor_data);
    cudaMemcpy(h_control_data, d_control_data, ... );
    update_motor_control(h_control_data);
}
```

What if my application frequently moves data between CPU and GPU?

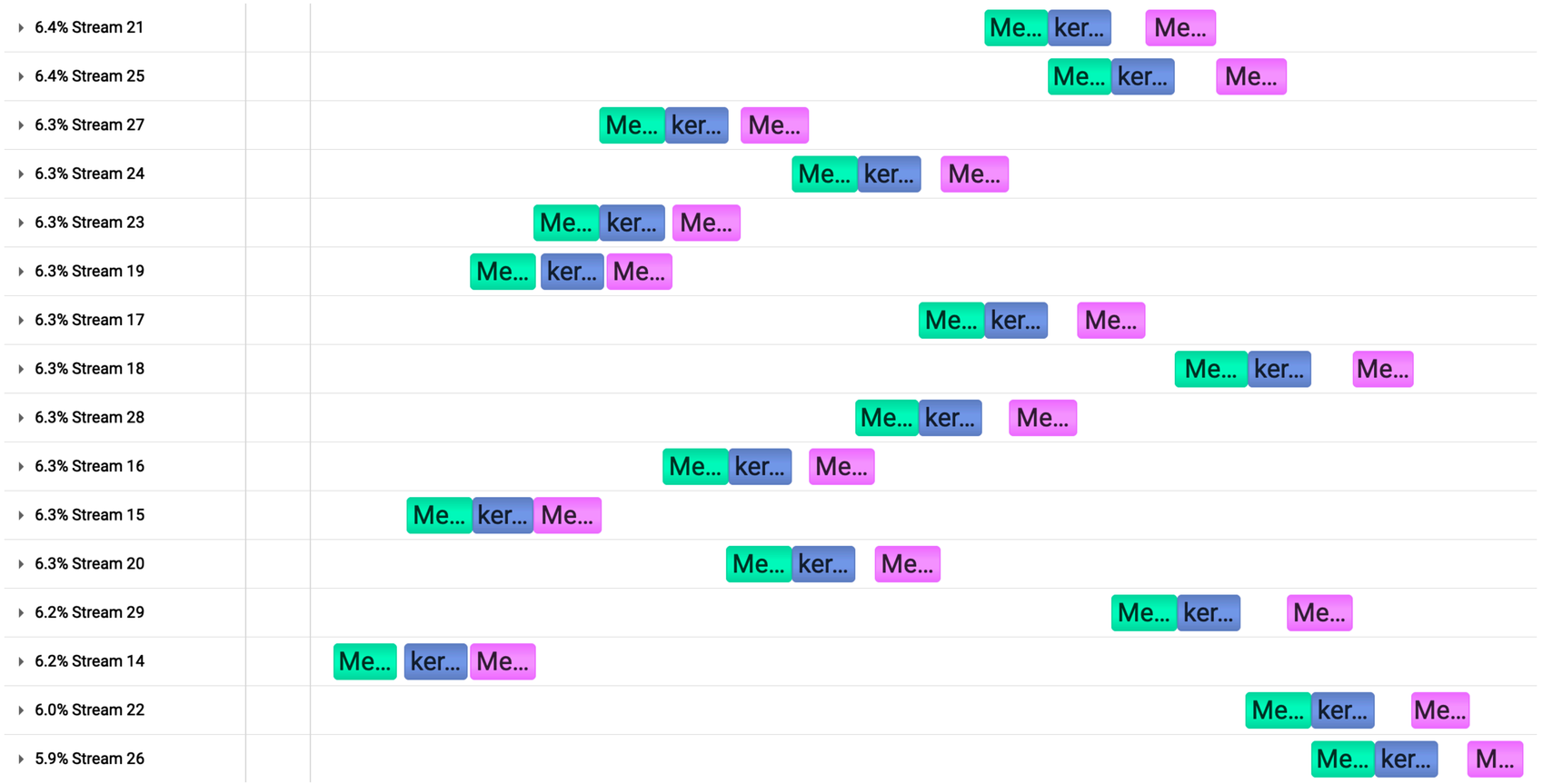
More CUDA Streams

Screenshot of timeline view in NVIDIA Nsight Systems



kernel execution can't start until cudaMemcpy finishes,
resulting in considerable idle time for GPU

More CUDA Streams



Instead, if we can process the problem in smaller pieces,

the data transfers can be overlapped with kernel execution

CUDA Stream Exercise 2:

Start with the baseline implementation in streams_for_data_movement.cu and modify it to overlap data movement and kernel execution by using streams.

- Allocate CPU buffers with `cudaMallocHost()`
- Create separate `cudaStream_t`'s
- Copy data with `cudaMemcpyAsync(dst, src, sz, kind, stream)`
- Launch with `kernel<<<grid, block, shmem, stream>>>(args...);`

- How does performance change when varying the number of streams?
- How does performance change when the kernel is shorter or longer?

CUDA Stream Exercise 2:

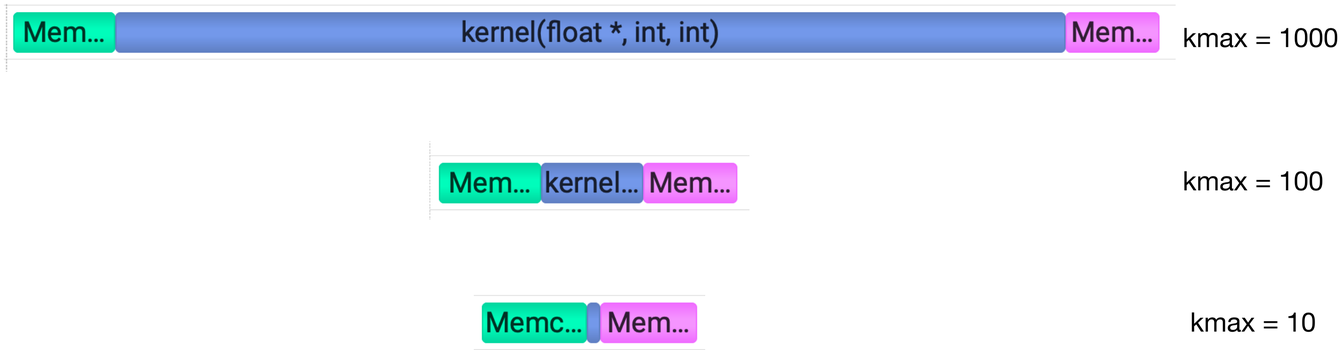
```
int chunk_size = n / num_streams;
int block = 256;
int grid = chunk_size / block;

for (int i = 0; i < num_streams; i++) {

    int offset = i * (n / num_streams);
    cudaMemcpyAsync(d_data + offset,
                    h_data + offset,
                    chunk_size * sizeof(float),
                    cudaMemcpyHostToDevice,
                    stream[i]);

    kernel<<< grid, block, 0, stream[i]>>>(d_data + offset, chunk_size);

    cudaMemcpyAsync(h_data + offset,
                    d_data + offset,
                    chunk_size * sizeof(float),
                    cudaMemcpyDeviceToHost,
                    stream[i]);

}
```

CUDA Stream Exercise 2:

streams	k_{max} = 10	k_{max} = 100	k_{max} = 1000
1	11.7873	16.1012	61.2766
2	8.85912	10.8004	56.0837
3	7.87159	9.20066	54.308
4	7.29841	8.25273	53.3963
5	7.02513	7.81512	52.8736
6	6.81761	7.45564	52.5164
7	6.76621	7.21427	52.2262
8	6.59543	7.00975	52.0409
9	6.56343	6.90174	51.8889
10	6.50742	6.81091	51.8087
11	6.46138	6.751	59.5166
12	6.43135	6.81398	51.6597
13	6.38457	6.7872	51.5251
14	6.3384	6.57314	51.5088
15	6.34715	6.78491	51.4527
16	6.34848	6.66907	51.4322

All times in milliseconds

```cpp
__global__ void kernel(float * data, int n, int kmax) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        float value = data[i];
        for (int k = 0; k < kmax; k++) {
            value = sin(value); // some expensive calculation
        }
        data[i] = value;
    }
}
```



all 3 images above are to-scale

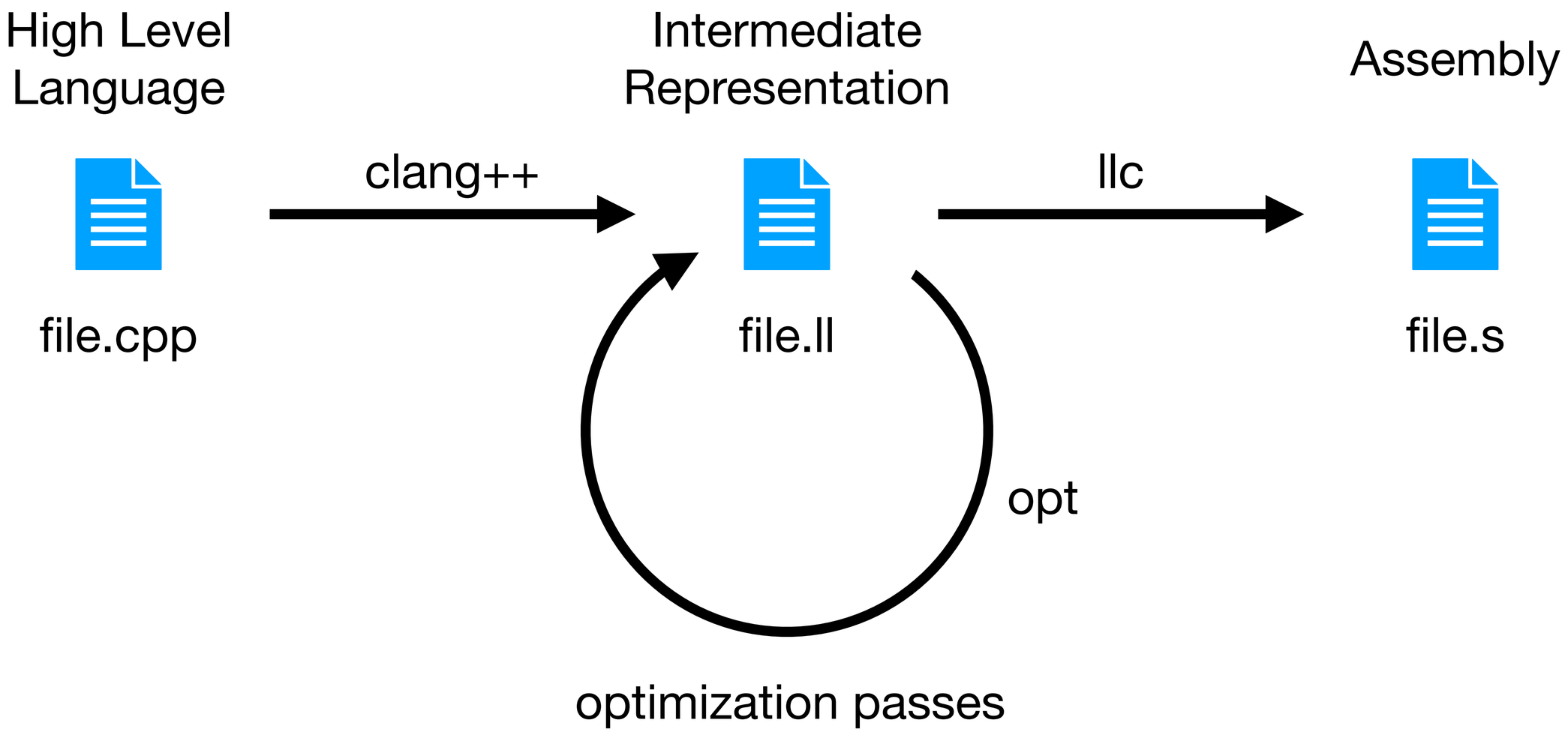
Review: Core CUDA Topics

- CPU vs GPU Hardware overview
- Serial vs SIMD vs SIMT programming models
- Memory Spaces
  - Memory Allocation (Host, Pinned, Device, Managed)
- CUDA kernels
  - registers
  - shared memory
  - occupancy
  - coalesced memory access
  - launch configuration and thread hierarchy
    - grid, block, warp, thread
  - __host__ / __device__ annotations
  - concurrency
    - atomics, synchronization, data races, divergence, determinism
  - stalls and latency hiding
- streams

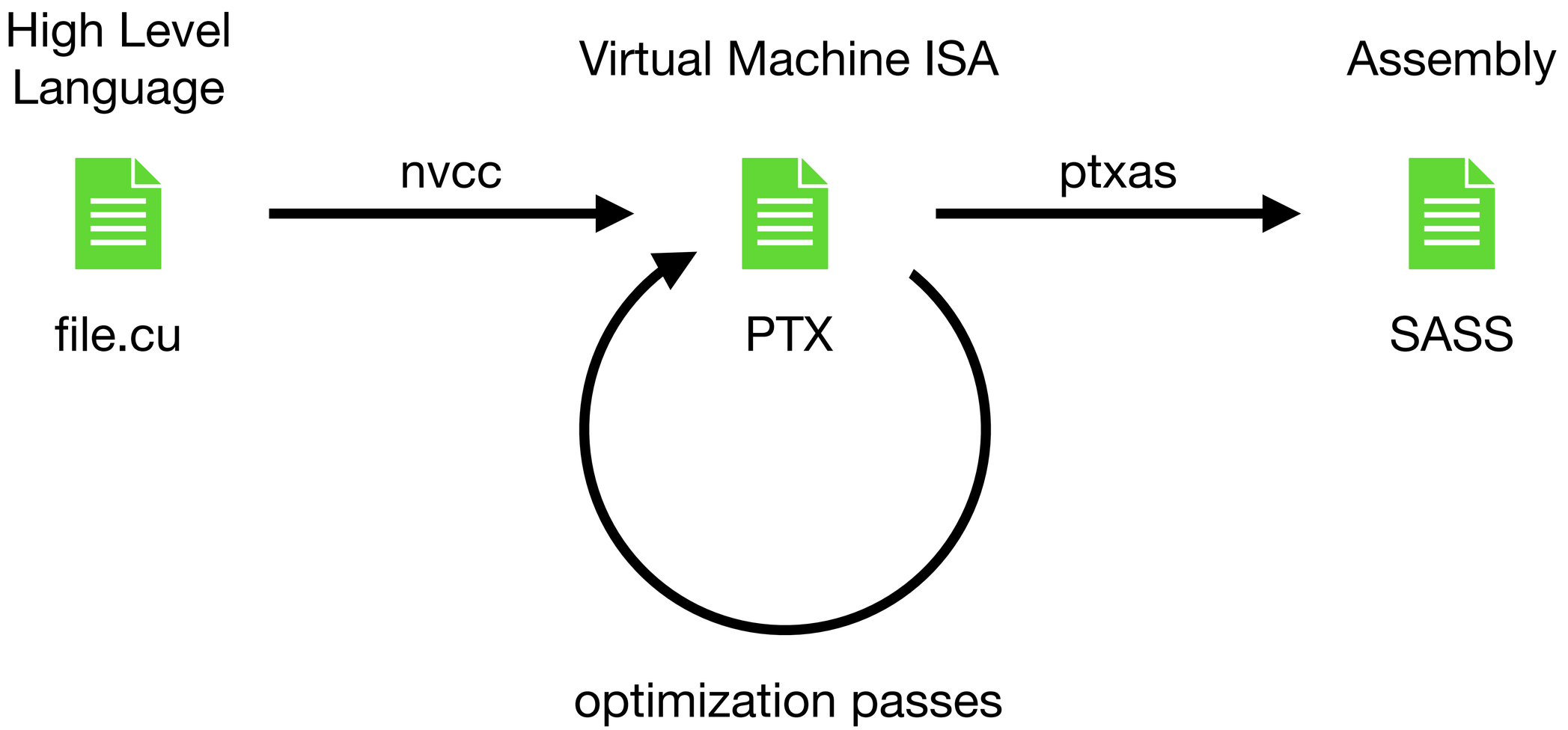
Session 3: Tools

- Compilers
  - nvcc, clang, nvc++
- Build
  - CMake, ninja / ninjatracing, perfetto
- Debugging
  - compute-sanitizer, cuda-gdb
- Profiling Tools
  - NSight Systems
  - NSight Compute
  - NVTX

Compilers


High Level Language
Intermediate Representation
Assembly
clang++
llc
file.cpp
file.ll
file.s
opt
optimization passes

Compilers


High Level Language
Virtual Machine ISA
Assembly
nvcc
ptxas
file.cu
PTX
SASS
optimization passes

Compilers

Demos:

- C++ to Intermediate Representation
- C++ to Assembly
- CUDA to PTX
- CUDA to SASS

See also: NVIDIA Tech Blog: Understanding PTX

Compilers

Some frequently useful nvcc flags:

- `--expt-relaxed-constexpr`
- `--extended-lambda`
- `-arch=native / -code`
- `-lineinfo`
- `--fdevice-time-trace=-` (CUDA 12.8+)

Compiler Explorer

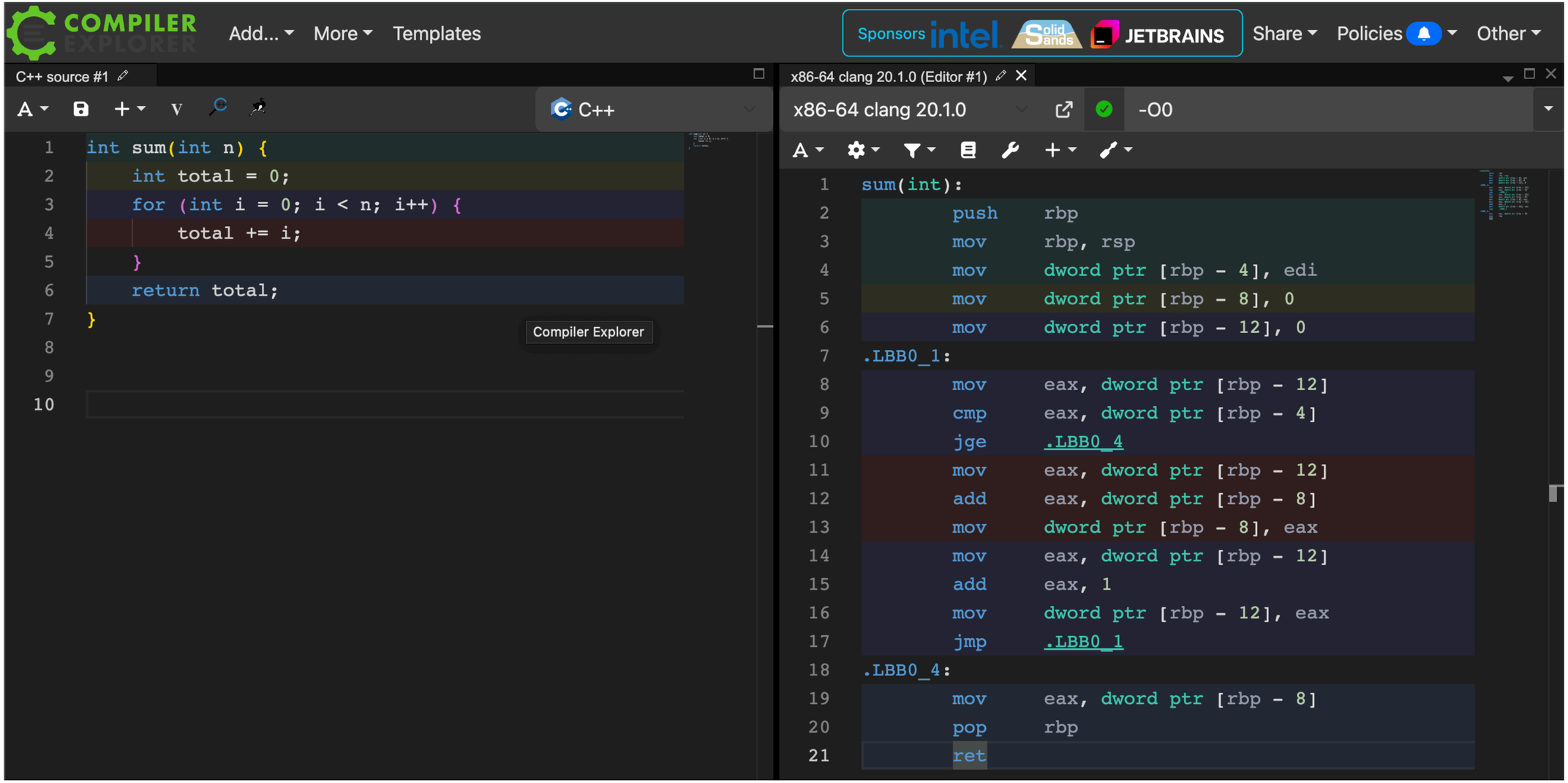
Useful for:

- learning / exploring new concept or feature
- sharing code snippets / bug reproducers
- examining generated assembly
- comparing compiler output
  - gcc vs. clang
  - different compiler versions
  - different compilation flags
- Prototyping ideas
- Trying out libraries

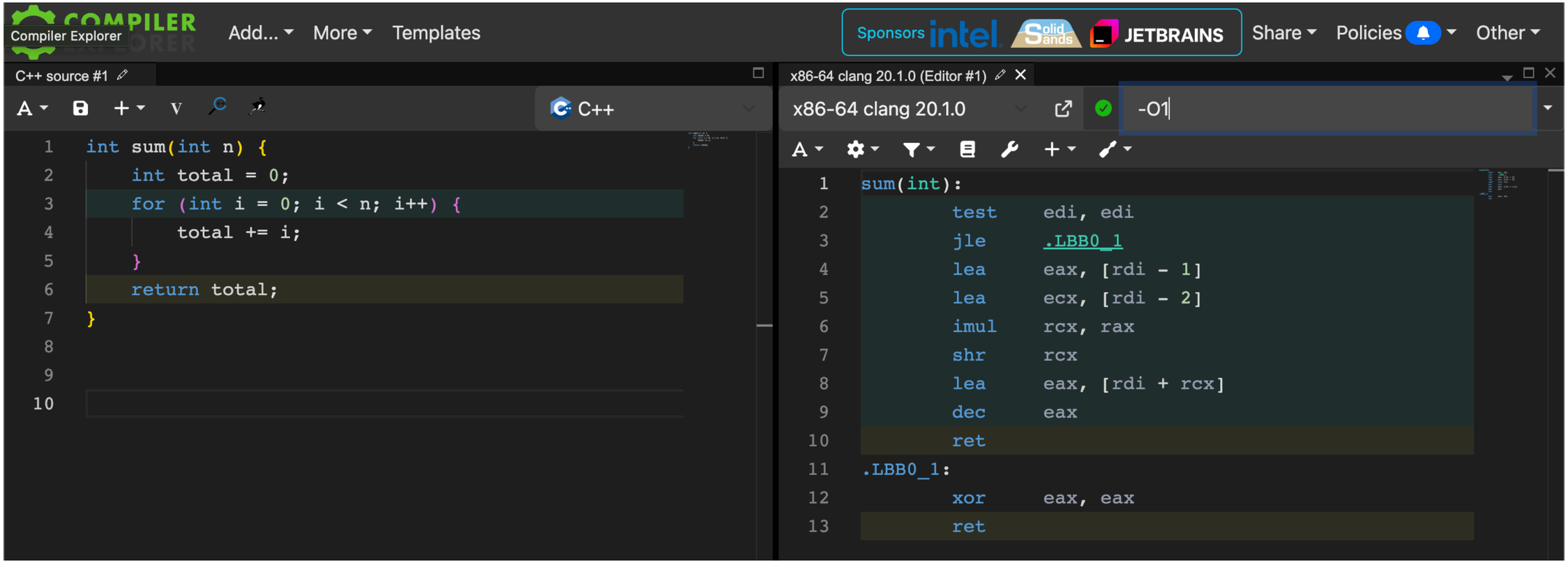
see also:

- "What has my compiler done for me lately?"
- NVIDIA Tech Blog: Compiler Explorer

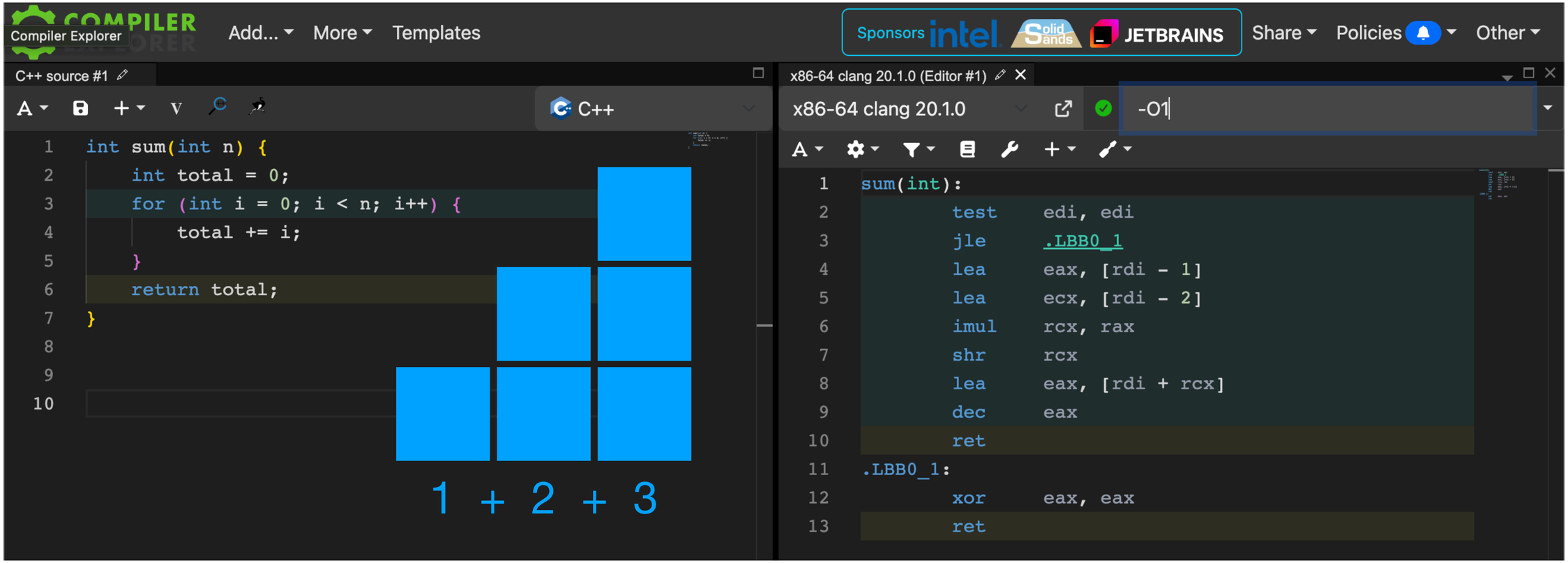
Fun CE Example



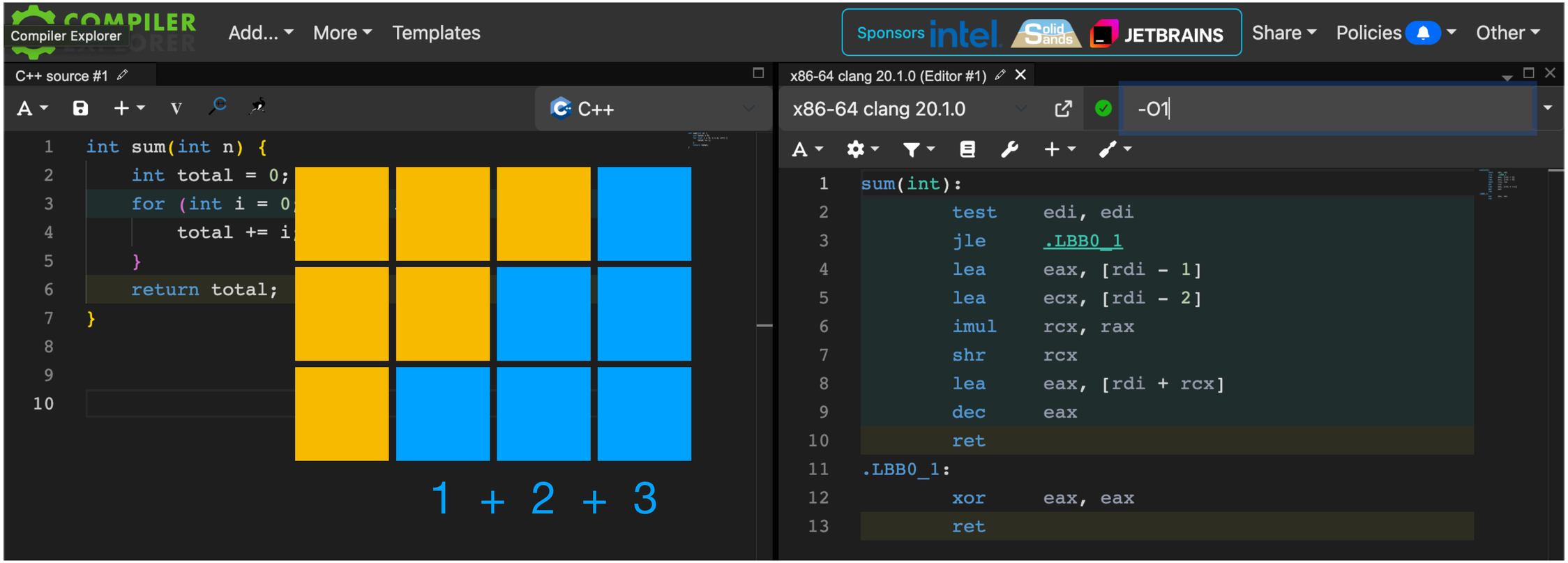
Fun CE Example



Fun CE Example



Fun CE Example



$$\overbrace{\sum_{i=1}^{n} i}^{\text{area of triangular staircase}} = \underbrace{\frac{1}{2}\, n\,(n+1)}_{\text{half the area of the rectangle}}$$

Compiler Explorer

Excerpt from "What's New and Important in CUDA Toolkit 13.0"

Updated vector types

`double4`, `long4`, `ulong4`, `longlong4`, and `ulonglong4` are vector data types originally introduced with 16-byte alignment. The Blackwell architecture introduces new support for 256-bit loads/stores. Using 32-byte alignment for these types can provide performance improvements for memory-bound applications.

To facilitate seamless use of these vector types with 32-byte aligned boundaries, we are adding additional vector types that explicitly specify the alignment. If you wish to use these vector types with 32-byte alignment, replace `double4` with `double4_32a`, for example. And if you wish to continue using these vector types with 16-byte alignment, replace `double4` with `double4_16a`, for instance.

Starting with CUDA 13.0, the use of the vector types `double4`, `long4`, `ulong4`, `longlong4`, and `ulonglong4` will include a deprecation warning encouraging you to replace these types with `_16a` or `_32a` variants, depending on your alignment requirements.

Compiler Explorer

Excerpt from "What's New and Important in CUDA Toolkit 13.0"

Earlier this year, I found a bug using compiler explorer
while having a discussion with a coworker about vectorized types.
Compiler explorer made it easy to show the unexpected SASS code,
and made it easy to share a working bug report with the compiler team

Compilation Time

```
sam@gouda:~/code/sandbox$ cat tmp.cpp
int main() { return 0; }
sam@gouda:~/code/sandbox$ time g++ tmp.cpp
real    0m0.066s
user    0m0.041s
sys     0m0.025s
sam@gouda:~/code/sandbox$ time clang++ tmp.cpp
real    0m0.069s
user    0m0.038s
sys     0m0.031s
sam@gouda:~/code/sandbox$ mv tmp.cpp tmp.cu && time nvcc tmp.cu
real    0m0.463s
user    0m0.340s
sys     0m0.123s
```

Compilation Time

```
sam@gouda:~/code/sandbox$ cat tmp.cpp
int main() { return 0; }
sam@gouda:~/code/sandbox$ time g++ tmp.cpp
real    0m0.066s
user    0m0.041s
sys     0m0.025s
sam@gouda:~/code/sandbox$ time clang++ tmp.cpp
real    0m0.069s
user    0m0.038s
sys     0m0.031s
sam@gouda:~/code/sandbox$ mv tmp.cpp tmp.cu && time nvcc tmp.cu
real    0m0.463s
user    0m0.340s
sys     0m0.123s
```

In practice, real examples are not actually 7x slower,
but compiling CUDA sources does usually
take longer than compiling C++ sources

Compile Time Tips

- separate code into .cpp vs .cu files
  - precompiled headers
  - fast linkers (lld / mold)
- compilation firewall / PIMPL idiom / `extern template`
- `--ftime-trace` (clang)
- `--fdevice-time-trace=-` (CUDA 12.8+)
- separable compilation / device LTO

CMake

- Native CUDA language support
  - Built-in targets for CUDA Toolkit libraries
  - Especially useful for RDC, device linking
- Helpful for managing dependencies
- Built-in testing framework
- Cross-platform builds
- `ninja` generator (see also: ninjatracing)

ninjabuild

```
sam@gouda:~/code/intro_to_cuda/build$ time ninja
real  0m3.604s
user  0m35.486s
sys   0m7.785s

sam@gouda:~/code/intro_to_cuda/build$ time make -j
real  0m4.361s
user  0m37.476s
sys   0m8.933s
```

Debugging

Methods:

- Error checking
- Print Statements
- `compute-sanitizer`
- `cuda-gdb`

Error Checking

- most CUDA Runtime API calls return an error code
  - kernel launches don't return an error code
  - errors from launches show up on next Runtime API call
    - e.g. cudaDeviceSynchronize() or cudaPeekAtLastError()
- simple to implement, no appreciable runtime cost
- often omitted from code examples for brevity
- for example (see error_checking.hpp):

```
double * d_pi_approx;
cudaMalloc(&d_pi_approx, sizeof(double));
cudaMemset(d_pi_approx, 0, sizeof(double));
```

```
#include "error_checking.hpp"

double * d_pi_approx;
CUDA_CHECK(cudaMalloc(&d_pi_approx, sizeof(double)));
CUDA_CHECK(cudaMemset(d_pi_approx, 0, sizeof(double)));
```

Print Statements

- `printf()` works inside a kernel or `__device__` function
- Note: `printf()` does incur a small cost in a kernel
  - requires a few extra registers to implement

```
__global__ void kernel() {

    // ...

    if (something_went_wrong) {
        printf("error message: %d\n", data);
    }

    // ...

}
```

`compute-sanitizer`

- Tool bundled with CUDA Toolkit installation
- Can catch a number of different kinds of bugs
  - basic usage: `compute-sanitizer ./my_program`
- has several different tools for finding memory leaks, accessing uninitialized data, data races, etc.
- Doesn't require any changes to executable
  - Although adding `lineinfo` is helpful
- Does significantly slow down kernel execution
  - Often not suitable for full-scale problems

compute-sanitizer

Exercise: check out the files 05_bugs/bug_[1-6].cu

Inspect the sources first and look for any obvious errors.

Then, try running the individual source files with `compute-sanitizer` with the flags indicated at the top of each source file.

Modify the sources as necessary until each bug is fixed

Note: bug_exercise.cu is intended to come after `cuda-gdb` but you can start looking at that one too, if you like

cuda-gdb

- Tool bundled with CUDA Toolkit installation
- Allows us to halt execution in host or device code and inspect temporaries, set watches, move around the call stack
- Requires "-G" flag to halt inside a kernel or device function
  - -G disables a lot of optimizations so kernel execution is much slower than normal
- basic usage: `cuda-gdb ./my_program`

compute-sanitizer + cuda-gdb

Exercise: check out the file 05_bugs/bug_exercise.cu

Inspect the sources first and look for any obvious errors. Try running compute-sanitizer and/or cuda-gdb to identify bugs and fix them.

Which bugs does compute-sanitizer find?
Which bugs does compute-sanitizer *not* find?
How does the runtime change with compute-sanitizer?
How does the runtime change when compiling with "-G"?

Debugging Summary

- Concurrency bugs can be tricky to debug, even with tools
  - determinism again
- The tools available (cuda-gdb, compute-sanitizer) are helpful
  - but still no "silver bullet"
- Debugging long-running applications is even harder
  - a combination of logging + unit test fixtures can help here**

** also my preferred approach for porting + optimizing kernels

Profilers

- NVIDIA provides two profilers with the CUDA Toolkit:
  - NSight systems for the "high level" profile of an application
  - NSight compute for the "low level" analysis of individual kernels
- Also, the library NVTX can be used to help annotate reports
  - NVTX is bundled with CUDA toolkit as well

NSight Systems

Demo

NSight Systems + NVTX

Demo

NSight Systems + NVTX

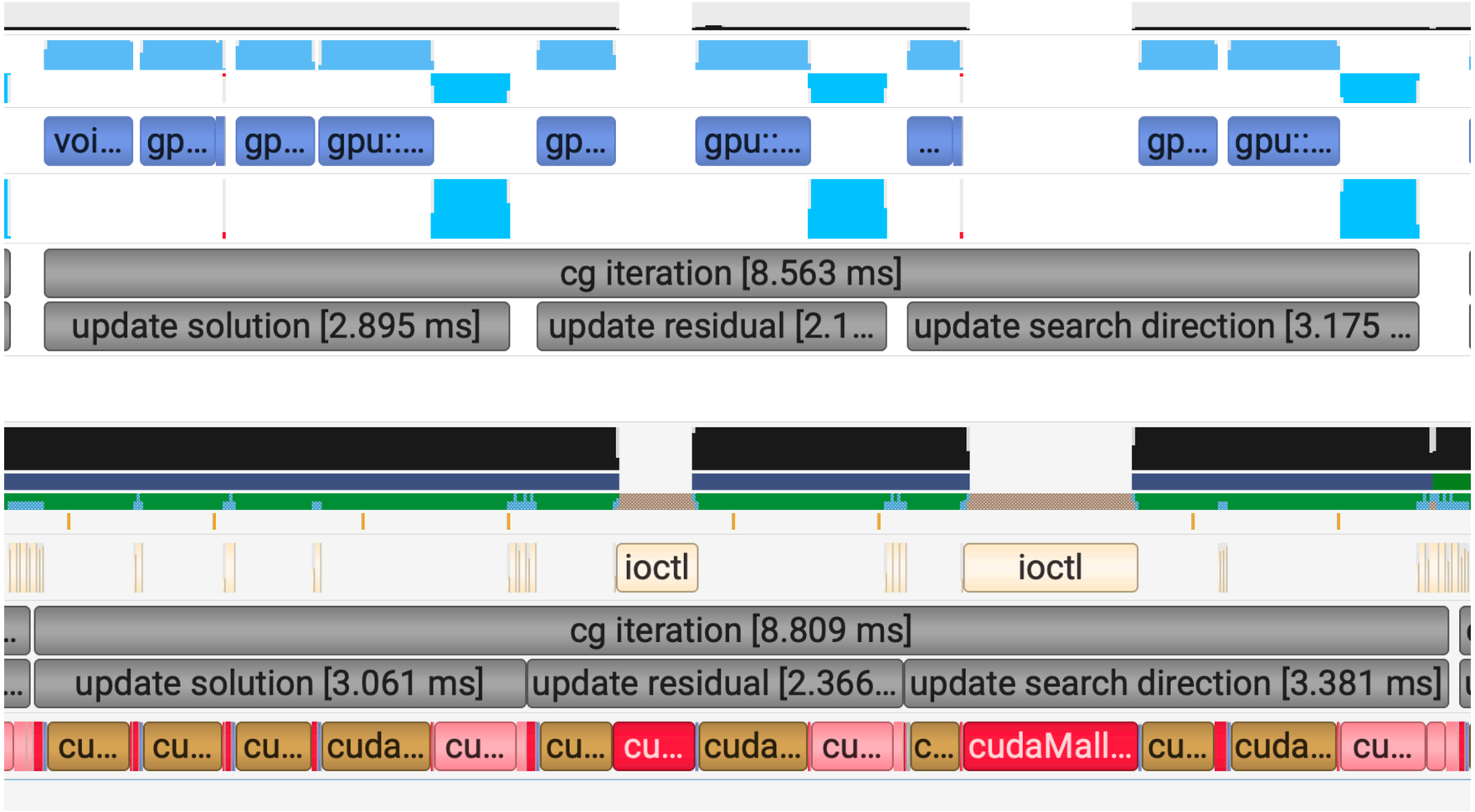
Exercise: look at the source files in 06_profiling/conjugate_gradient

Build the executable and profile it with NSight systems.

Insert NVTX annotations into the conjugate gradient implementation krylov.hpp (see file for more detailed instructions) and profile it again with NSight systems.

Inspect the report with the NSight Systems UI.
What fraction of the runtime is spent in each kernel?

NSight Systems + NVTX



NSight Systems + NVTX

Stats System View

- CUDA API Summary
- CUDA API Trace
- CUDA GPU Kernel Summary
- CUDA GPU Kernel/Grid/Block Summary
- CUDA GPU MemOps Summary (by Size)
- CUDA GPU MemOps Summary (by Time)
- CUDA GPU Summary (Kernels/MemOps)
- CUDA GPU Trace
- CUDA Kernel Launch & Exec Time Summa
- CUDA Kernel Launch & Exec Time Trace
- CUDA Summary (API/Kernels/MemOps)
- DX11 PIX Range Summary
- DX12 GPU Command List PIX Ranges Sur
- DX12 PIX Range Summary
- MPI Event Summary

Time	Total Time	Instances	Avg	Med	Min	Max	StdDev	Name
29.2%	71.421 ms	150	476.138 µs	476.383 µs	446.399 µs	478.527 µs	3.340 µs	gpu::scale(double *, const double *, double, int)
28.2%	69.006 ms	100	690.056 µs	690.447 µs	651.839 µs	691.359 µs	3.879 µs	gpu::add(double *, double *, double *, int)
15.8%	38.753 ms	101	383.696 µs	296.928 µs	287.551 µs	479.935 µs	91.925 µs	gpu::dot_1(const double *, const double *, int, double *)
14.1%	34.522 ms	50	690.438 µs	690.414 µs	689.375 µs	691.327 µs	380 ns	gpu::sub(double *, double *, double *, int)
11.4%	27.905 ms	51	547.153 µs	547.807 µs	511.551 µs	549.503 µs	5.154 µs	void laplace_operator<double>(span<(int)3, T1>, span<(int)3, T1>)
1.2%	2.958 ms	101	29.282 µs	29.248 µs	28.192 µs	30.336 µs	390 ns	gpu::dot_2(int, double *, double *)

NSight Compute

Demo

NSight Compute

Exercise: look at the source file in 06_profiling/unstructured_mesh.cu

Write CUDA kernels for the two indicated CPU calcuations

Profile with NSight Compute and comment on the performance bottlenecks for the two different numberings. How does the memory traffic differ in each case?